32-bit Modular Arithmetic and SIMD vectorization

Duc Vinh Nguyen, Henry Zheng

March 2026

SORBONNE UNIVERSITÉ

Contents

1 Introduction **2**

2 Fundamental Cost of Modular Reduction **2**

3 Shoup's Modular Multiplication **2**

3.1 Input Constraint: . . . 2
3.2 Theorem . . . 3
3.3 Input Hypothesis: . . . 3
3.4 Precomputational Logic (Barrett Reduction) . . . 3
3.5 Fundamental Principle . . . 4
3.6 Demonstration: . . . 4
3.7 Advantages . . . 5
3.8 SIMD vectorization Implementation . . . 5
3.8.1 AVX2 . . . 5
3.8.2 NEON . . . 10
3.9 Performance . . . 11
3.9.1 Equipement detail: . . . 11
3.9.2 Choice of unrolling . . . 11
3.9.3 Choice between mullo_epi32 and mul_epu32 . . . 13
3.9.4 Versions of mullo: . . . 15
3.9.5 Overall Performance . . . 15
3.9.6 NEON's Performance . . . 17
3.10 Overall Conclusion: . . . 17
3.11 Other Improvements: . . . 17

1 Introduction

This project explores and optimizes modular reductions modulo primes $p < 2^{32}$ using SIMD vectorization (AVX2, AVX512, NEON) along with various techniques and algorithms.

Objective: The aim of this study is to develop an optimized implementation of modular reduction for 32-bit primes, minimizing execution time through SIMD vectorization.

2 Fundamental Cost of Modular Reduction

The naive approach to computing $a \pmod{n}$ is to evaluate:

$$a - \left\lfloor \frac{a}{n} \right\rfloor \cdot n \tag{1}$$

However, on modern processor architectures (Intel, AMD,...), integer division has a significantly higher computational cost compared to basic arithmetic operations such as additions and multiplications.

Since equation (1) requires an integer divsion, a multiplication and a substraction operation, its considered highly inefficient from both a software and hardware perspective.

3 Shoup's Modular Multiplication

Shoup's algorithm is based on precomputation to avoid costly operations.

Algorithm 1: Shoup's modular multiplication algorithm

Input : $A, B \in [0, p[$, we suppose that $p < \beta/2$
precompute $B' = \lfloor B\beta/p \rfloor$

Output: $R = AB \bmod p$

$Q \leftarrow \lfloor AB'/\beta \rfloor$;
$R \leftarrow (AB - Qp) \bmod \beta$;
if $R \geq p$ **then**
  $R \leftarrow R - p$;

3.1 Input Constraint:

To ensure the correctness of the modular reduction and prevent register overflow during SIMD operations, the following conditions must be satisfied:

- **Word Size (β):** Set to 2^{32} for 32-bit modular arithmetic
- **Input 1 (A):** An integer defined over 32 bits
- **Input 2 (B):** Integer within the range $[0, p[$
- **Modulus (p):** A prime number defined over 31 bits ($p < 2^{31}$)

3.2 Theorem

Let p be a prime number such that $p < \beta/2$ where $\beta = 2^{32}$. Given A an integer defined over 32 bits and $B \in [0, p)$, let B' be the precomputed value $B' = \lfloor B\beta/p \rfloor$. If the estimated quotient is defined as $Q = \lfloor AB'/\beta \rfloor$, then the intermediate remainder $R = AB - Qp$ satisfies the following condition:

$$0 \leq AB - Qp < 2p$$

3.3 Input Hypothesis:

Can p be any integer over 32 bits ?
No, one particularly interesting case where Shoup's algorithm will fail is for $p \geq 2^{31}$. Since we know that:

$$0 \leq AB - Qp < 2p$$

If $p \geq 2^{31}$, thes $2p \geq 2^{32}$. Therefore, in some cases $AB - Qp$ may exceed 2^{32}. This causes tremendous errors in the calculation because all bits beyond 32 bits are destroyed when we reduce modulo β

Does p have to necessarily be a prime number ?
No, the algorithm mainly uses precomputations (Barrett Reduction), optimized for caculation of $a \mod n$ without constraints on n. But for the sake of this project, n is a prime number (p).

Can b be any integer over 32 bits ?
No, suppose $b > p$, we can rewrite b as:

$$b = q' \cdot p + r'$$

Thus the caculations for the precomputed value B' is:

$$B' = \left\lfloor \frac{(q' \cdot p + f) \cdot 2^{32}}{p} \right\rfloor$$

$$B' = q' \cdot 2^{32} + \left\lfloor \frac{f \cdot 2^{32}}{p} \right\rfloor$$

With a focus on 32 bits in this project, B' in the case of $b > p$ cannot be stored within 32 bits and will overflow.

Can a be any integer over 32 bits ?
Yes, a can be any integer defined over 32 bits.

3.4 Precomputational Logic (Barrett Reduction)

The precomputed value B' is defined over 32 bits as:

$$B' = \lfloor (B \cdot 2^{32})/p \rfloor$$

When computing the $a \cdot b \pmod p$:

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot B}{p} \right\rfloor \cdot p$$

We can rewrite this:

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot \left\lfloor \left(\frac{B \cdot 2^{32}}{p}\right) \right\rfloor}{2^{32}} \right\rfloor \cdot p$$
$$\approx A \cdot B - \left\lfloor \frac{A \cdot B'}{2^{32}} \right\rfloor \cdot p$$

3.5 Fundamental Principle

Given x the quotient of 2 32-bit integers:
The proof of this algorithm is based on the non-decimal value of ($\lfloor x \rfloor$):

$$x - 1 < \lfloor x \rfloor \leq x$$

We can rewrite as:

$$0 \leq x - \lfloor x \rfloor < 1$$

3.6 Demonstration:

The algorithm uses the precomputation $B' = \lfloor B\beta/p \rfloor$ such that:

$$0 \leq \frac{B\beta}{p} - \left\lfloor \frac{B\beta}{p} \right\rfloor < 1$$
$$0 \leq \frac{B\beta}{p} - B' < 1 \tag{2}$$

Using the same logic, the algorithm estimates the quotient $Q = \lfloor AB'/\beta \rfloor$ such that:

$$0 \leq \frac{AB'}{\beta} - \left\lfloor \frac{AB'}{\beta} \right\rfloor < 1$$
$$0 \leq \frac{AB'}{\beta} - Q < 1 \tag{3}$$

We now prove that the remainder $R = (A \cdot B - Q \cdot p)$ lies within the interval $[0, 2p[$:

We firstly multiply the equation (1) by Ap/β:

$$0 \leq \left(\frac{B\beta}{p} - B'\right) \cdot \frac{Ap}{\beta} < 1 \cdot \frac{Ap}{\beta}$$
$$0 \leq AB - \frac{B'Tp}{\beta} < \frac{Ap}{\beta} \tag{4}$$

Similarly, we multiply the equation (2) by p:

$$0 \leq \left(\frac{AB'}{\beta} - Q\right) \cdot p < 1 \cdot p$$

$$0 \leq \frac{AB'p}{\beta} - Qp < p \tag{5}$$

By adding the equation (3) and (4), we deduce:

$$0 \leq (AB - \cancel{\frac{AB'p}{\beta}}) + (\cancel{\frac{AB'p}{\beta}} - Qp) < \frac{Ap}{\beta} + p$$

Simplifying this result, we are left with:

$$0 \leq AB - Qp < p + \frac{Ap}{\beta}$$
$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta})$$

By definition, $A \in [0, p[$ and $p < \beta$, thus $A < \beta$. Therefore we can write:

$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta}) < p \cdot (1 + \frac{\beta}{\beta})$$
$$0 \leq AB - Qp < p \cdot (1 + 1)$$
$$0 \leq AB - Qp < 2p$$
$$0 \leq R < 2p$$

Conclusion: We have now proven that $R \in [0, 2p[$. The final if statement of the algorithm helps correct the remainder: if $R > p$, one more step $R \leftarrow R - p$ is needed; if not, R is the correct result.

3.7 Advantages

Using the precomputed value B' to approximately calculate $\left\lfloor \frac{A \cdot B}{p} \right\rfloor$ improves upon on the naive method into two ways:

1. **Storing the precomputed value:** B' is computed with one division and can be stored for use in other modular reductions, provided the same B and the same p are used.
2. **Efficient operations:** Shoup's algorithm replaces slow division operations with fast multiplications and bit shifts. The expression $\frac{A \cdot B'}{2^{32}}$ requires only one multiplication operation and a right bit shift by 32 bits. On a CPU, both multiplication and bit shift operations are significantly more efficient than division.

3.8 SIMD vectorization Implementation

We decided to create two versions for implementing a high-performance Shoup algorithm.

3.8.1 AVX2

For the AVX2 version, we decided to use two main AVX2 functions: `_mm256_mul_epu32` and `_mm256_mullo_epi32`.

Intel `_mm256_mul_epu32`
This function treats the input as unsigned 32-bit integers. However, it only multiplies the even-indexed slots (0, 2, 4, 6) of the vector to make room for the full 64-bit results.

Listing 1: Auxilary Function of Shoup's Modular Reduction

```
static inline __m256i _shoup_avx2(__m256i va,
                                  __m256i vb,
                                  __m256i vb_precomp,
                                  __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // vab = [ (a_0 * b_0 = ab_0ab_1), (a_2 * b_2 = ab_2ab_3),
    // (a_4 * b_4 = ab_4ab_5), (a_6 * b_6 = ab_6ab_7) ]
    __m256i vab = _mm256_mul_epu32(va, vb);

    // vqp = [ (q_0 * p_0 = qp_0qp_1), (q_2 * p_2 = qp_2qp_3),
    // (q_4 * p_4 = qp_4qp_5), (q_6 * p_6 = qp_6qp_7) ]
    __m256i vqp = _mm256_mul_epu32(vq, vp);

    // vc = [ (ab_0 - qp_0 = c_0c_1), (ab_2 - qp_2 = c_2c_3),
    // (ab_4 - qp_4 = c_4c_5), (ab_6 - qp_6 = c_6c_7) ]
    __m256i vc = _mm256_sub_epi64(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 2: AVX2 Implementation of Shoup's Modular Reduction

```
Vector shoup_scale_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        // Load [ a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7 ]
        //    = [ (a_1, a_0), (a_3, a_2), (a_5, a_4), (a_7, a_6) ]
        // but only even index will be directly handled
        __m256i even_va = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i)
        );

        // Need to move odd index to even index to be handled:
        // [ (0, a_1), (0, a_3), (0, a_5), (0, a_7) ]
        __m256i odd_va = _mm256_srli_epi64(even_va, 32);

        // Process Even Lanes
        __m256i even_vc = _shoup_avx2(even_va, vb, vb_precomp, vp);

```

```
        // Process Odd Lanes:
        __m256i odd_vc = _shoup_avx2(odd_va, vb, vb_precomp, vp);

        // Shift odd by 32 bits to the left results back and merge:
        // [ (c_1, 0), (c_3, 0), (c_5, 0), (c_7, 0) ]
        __m256i odd_shifted = _mm256_slli_epi64(odd_vc, 32);

        // Merge even and odd: [ c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7 ]
        __m256i merge = _mm256_or_si256(even_vc, odd_shifted);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Intel `_mm256_mullo_epi32`
This function treats the input as signed 32-bit integers. It multiplies all 8 pairs of numbers but only stores the bottom half of the result.

Listing 3: Auxilary Function of Shoup's Modular Reduction (Version mullo)

```
static inline __m256i _shoup_mullo_avx2(__m256i va,
                                        __m256i vb,
                                        __m256i vb_precomp,
                                        __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // Preserves only the least significant bit:
    // vab = [ (a_0 * b_0 mod 2^32 = ab_0), (a_2 * b_2 mod 2^32 = ab_2),
    // (a_4 * b_4 mod 2^32 = ab_4), (a_6 * b_6 mod 2^32 = ab_6) ]
    __m256i vab = _mm256_mullo_epi32(va, vb);

    // vqp = [ [q_0 * p_0 mod 2^32 = qp_0], [q_2 * p_2 mod 2^32 = qp_2],
    // [q_4 * p_4 mod 2^32 = qp_4], [q_6 * p_6 mod 2^32 = qp_6] ]
    __m256i vqp = _mm256_mullo_epi32(vq, vp);

    // vc = [ (0, ab_0 - qp_0), (0, ab_2 - qp_2), (0, ab_4 - qp_4),
    // (0, ab_6 - qp_6) ]
    __m256i vc = _mm256_sub_epi32(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 4: AVX2 Implementation of Shoup's Modular Reduction (Version mullo)

```
Vector shoup_scale_mullo_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 31 < size; i += 32)
    {
        __m256i even_va_0 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i));
        __m256i even_va_1 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 8));
        __m256i even_va_2 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 16));
        __m256i even_va_3 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 24));

        __m256i odd_va_0 = _mm256_srli_epi64(even_va_0, 32);
        __m256i odd_va_1 = _mm256_srli_epi64(even_va_1, 32);
        __m256i odd_va_2 = _mm256_srli_epi64(even_va_2, 32);
        __m256i odd_va_3 = _mm256_srli_epi64(even_va_3, 32);
```

```

        __m256i even_vc_0 = _shoup_mullo_avx2(even_va_0, vb, vb_precomp, vp);
        __m256i even_vc_1 = _shoup_mullo_avx2(even_va_1, vb, vb_precomp, vp);
        __m256i even_vc_2 = _shoup_mullo_avx2(even_va_2, vb, vb_precomp, vp);
        __m256i even_vc_3 = _shoup_mullo_avx2(even_va_3, vb, vb_precomp, vp);

        __m256i odd_vc_0 = _shoup_mullo_avx2(odd_va_0, vb, vb_precomp, vp);
        __m256i odd_vc_1 = _shoup_mullo_avx2(odd_va_1, vb, vb_precomp, vp);
        __m256i odd_vc_2 = _shoup_mullo_avx2(odd_va_2, vb, vb_precomp, vp);
        __m256i odd_vc_3 = _shoup_mullo_avx2(odd_va_3, vb, vb_precomp, vp);

        __m256i odd_shifted_0 = _mm256_slli_epi64(odd_vc_0, 32);
        __m256i odd_shifted_1 = _mm256_slli_epi64(odd_vc_1, 32);
        __m256i odd_shifted_2 = _mm256_slli_epi64(odd_vc_2, 32);
        __m256i odd_shifted_3 = _mm256_slli_epi64(odd_vc_3, 32);

        __m256i merge_0 = _mm256_or_si256(even_vc_0, odd_shifted_0);
        __m256i merge_1 = _mm256_or_si256(even_vc_1, odd_shifted_1);
        __m256i merge_2 = _mm256_or_si256(even_vc_2, odd_shifted_2);
        __m256i merge_3 = _mm256_or_si256(even_vc_3, odd_shifted_3);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge_0);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 8), merge_1);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 16), merge_2);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 24), merge_3);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Intel _mm256_mullo_epi32 version 2:
Derived from the previous version, this version aims to regroup the vector registers immediately after the multiplication step

$$q = \lfloor (a \cdot b_{precomp})/2^{32} \rfloor$$

By restoring the standard 8-lane 32-bit layout early, the function eliminates the need to perform parallel logic paths. This, in theory, should reduce the work load.

In order to implement this version, we create an auxilary function **_mulhi_emul_avx2** which emulates a native 'multiply-high' instruction for unsigned 32-bit integers. It computes the full 64-bit products of two packed vectors, extracts the most significant 32 bits from each result, and repacks them into a single 256-bit destination register.

Listing 5: Auxilary Function _fake_mulhi

```
static inline __m256i _mulhi_emul_avx2(__m256i va, __m256i vb)
{
    // res_even = [ (a_0 * b_0), (a_2 * b_2), (a_4 * b_4), (a_6 * b_6) ]
    __m256i res_even = _mm256_mul_epu32(va, vb);

    // res_odd = [ (a_1 * b_0), (a_3 * b_2), (a_5 * b_4), (a_7 * b_6) ]
    __m256i res_odd = _mm256_mul_epu32(_mm256_srli_epi64(va, 32), vb);

```

```
    // hi_even = [ (0, hi(a_0 * b_0)), (0, hi(a_2 * b_2)), (0, hi(a_4 * b_4)),
    // (0, hi(a_6*b_6)) ]
    __m256i hi_even = _mm256_srli_epi64(res_even, 32);

    // Removes the least significant bits: hi_odd = [ (hi(a_1 * b_0), 0),
    // (hi(a_3 * b_2), 0), (hi(a_5 * b_4), 0), (hi(a_7 * b_6), 0) ]
    __m256i hi_odd = _mm256_and_si256(res_odd, _mm256_set1_epi64x(
        0xFFFFFFFF00000000ULL));

    // [ (hi(a_1 * b_0), hi(a_0 * b_0)), (hi(a_3 * b_2), hi(a_2 * b_2)),
    // (hi(a_5 * b_4), hi(a_4 * b_4)), (hi(a_7 * b_6), hi(a_6 * b_6)) ]
    return _mm256_or_si256(hi_even, hi_odd);
}
```

Listing 6: Auxilary Function of Shoup's Modular Reduction (Version 2 mullo)

```
static inline __m256i _shoup_mullo_v2_avx2(__m256i va,
                                           __m256i vb,
                                           __m256i vb_precomp,
                                           __m256i vp)
{
    __m256i vq = _mulhi_emul_avx2(va, vb_precomp);
    __m256i vab = _mm256_mullo_epi32(va, vb);
    __m256i vqp = _mm256_mullo_epi32(vq, vp);
    __m256i vc = _mm256_sub_epi32(vab, vqp);
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 7: AVX2 Implementation of Shoup's Modular Reduction (Version 2 mullo)

```
Vector shoup_scale_mullo_v2_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi32(param.b);
    __m256i vb_precomp = _mm256_set1_epi32(param.b_precomp);
    __m256i vp = _mm256_set1_epi32(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        __m256i va = _mm256_loadu_si256((__m256i const *)(v.elements + i));

        __m256i vc = _shoup_mullo_v2_avx2(va, vb, vb_precomp, vp);
        _mm256_storeu_si256((__m256i *)(res.elements + i), vc);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

3.8.2 NEON

Listing 8: Auxilary Function of Shoup's Modular Reduction (Version Neon)

```
    TODO
```

Listing 9: NEON Implementation of Shoup's Modular Reduction

```
    TODO
```

3.9 Performance

3.9.1 Equipement detail:

The performance measurements and benchmarks were performed with the following hardware:

- Processor: AMD Ryzen™ 5 8600G Desktop Processor.
- Architecture: Zen 4 (supporting AVX, AVX2, and AVX-512 instruction sets)
- Compiler : GCC 13.3.0 with optimization flag -O3
- Execution : ./pcca7 -b_bits 31 (Option to set b to 31 bits)

3.9.2 Choice of unrolling

One way to optimize for faster functions through vectorization is Loop Unrolling. This technique consists of explanding the body of a loop to process multiple iterations at once.

The results below are obtained by measuring the execution time between a standard version and unrolled version of Listing 2: AVX2 Implementation of Shoup's Modular Reduction

Hypothesis: The unrolled version should perform better than the standard version

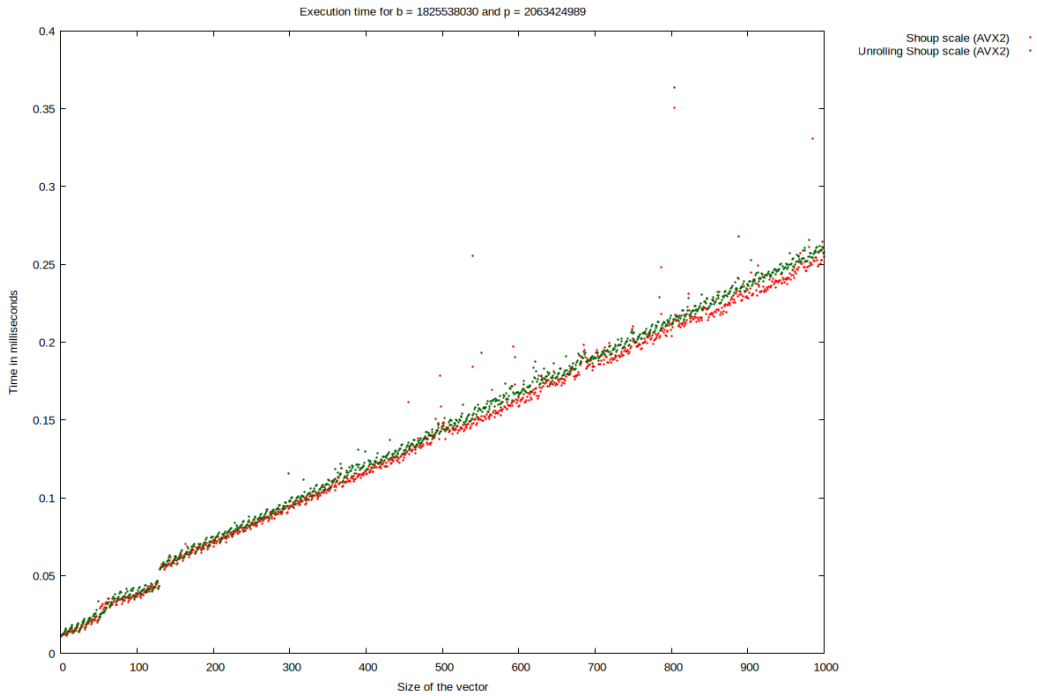

Figure 1: Execution Time Comparison: Standard vs. Unrolled

Table 1: Execution time of implementations with and without unrolling in milliseconds

Size	Standard	Unrolled
100	0.037	0.039
200	0.069	0.071
300	0.093	0.098
400	0.115	0.118
500	0.148	0.146
600	0.159	0.166
700	0.195	0.190
800	0.204	0.212
900	0.228	0.237
1000	0.255	0.258

Table 2: Average percentage speedup of the unrolling implementation relative to with unrolling

Size	Standard	Unrolled	Percentage difference
100	100%	105.41%	+5.41%
200	100%	102.90%	+2.90%
300	100%	105.38%	+5.38%
400	100%	102.61%	+2.61%
500	100%	98.65%	−1.35%
600	100%	104.40%	+4.40%
700	100%	97.44%	−2.56%
800	100%	103.92%	+3.92%
900	100%	103.95%	+3.95%
1000	100%	101.18%	+1.18%

Observation: On average, the Unrolled implementation is 2.58% slower than the standard version.

Conclusion: We expected to see a bigger performance gap favoring the unrolled version but from the results obtained, this doesnt seem to be the case. We can conclude that an unrolled version for our implmentation of Shoup's Modular Multiplication is not favorable.

3.9.3 Choice between mullo_epi32 and mul_epu32

mullo_epi32 and mul_epu32 both produces at max 64-bit results, the key difference is the former operation only truncates the results and only stores the 32 lower bits of the results whereas the later stores the full 64-bit results.

The results below are obtained by measuring the execution time between the 2 variantions described in Section 3.8.1

Hypothesis: The variant mullo_epi32 should perform better than the variant mul_epu32 as it uses less operations overall

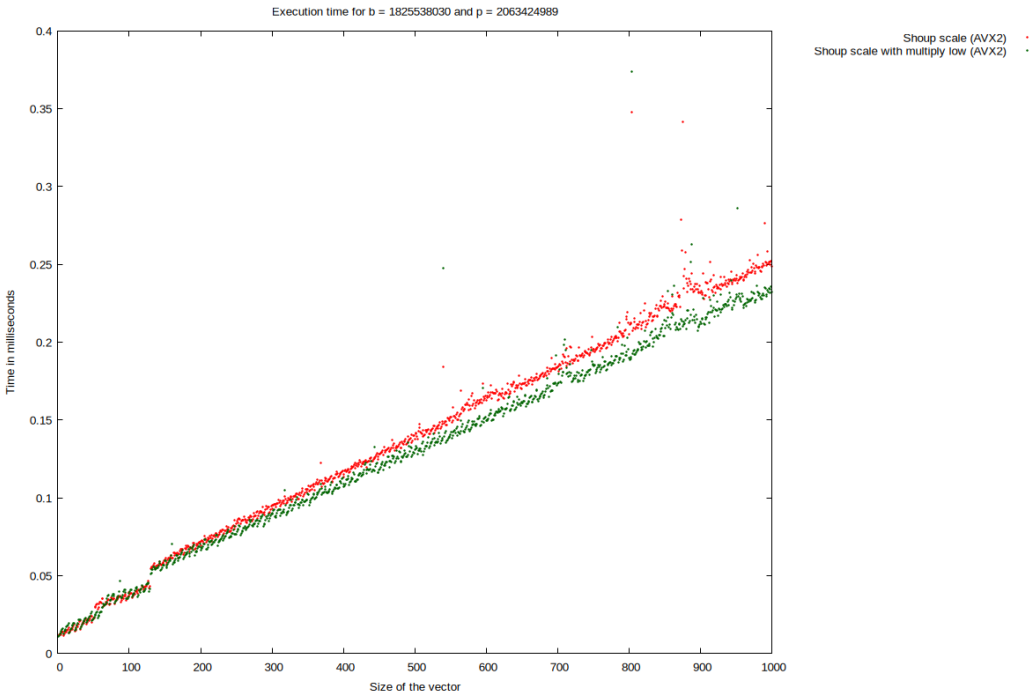

Figure 2: Execution Time Comparison: mullo_epi32 vs. mul_epu32

Table 3: Execution time of mullo_epi32 and mul_epu32 implementation in milliseconds

Size	mul_epu32	mullo_epi32
100	0.037	0.038
200	0.069	0.066
300	0.093	0.090
400	0.113	0.107
500	0.138	0.131
600	0.163	0.150
700	0.184	0.173
800	0.205	0.190
900	0.232	0.213
1000	0.249	0.232

Table 4: Average percentage speedup of mullo_epi32 implementation relative to mul_epu32

Size	mul_epu32	mullo_epi32	Percentage difference
100	100%	102.70%	+2.70%
200	100%	95.65%	−4.35%
300	100%	96.77%	−3.23%
400	100%	94.69%	−5.31%
500	100%	94.93%	−5.07%
600	100%	92.02%	−7.98%
700	100%	94.02%	−5.98%
800	100%	92.68%	−7.32%
900	100%	91.81%	−8.19%
1000	100%	93.17%	−6.83%

Observation: We can observe that at the smaller sizes (100), mullo_epi32 is slower than the standard mul_epu32 implementation by around 2%. But the effecientcy gain starts to show once the vector size hits 200 and above. Across all the vector sizes, the average percentage gain is 5.15%

Conclusion: The mullo_epi32 version is effectively faster than the mul_epu32 version by about 5%, this speedup is especially seen for bigger vectors sizes. The mullo_epi32 version is the better implementation

3.9.4 Versions of mullo:

Both versions use mullo as the main multiplication operation, the key difference between these 2 versions is when the regrouping happens for the vector registers.

The results bellow are obtained by measuring the execution time between 2 implementations using _mm256_mullo_epi32 described in Section 3.8.1

Hypothesis: The second version implementation should perform better, it carries out les redundant code by regrouping early the vector registers

Observations:

Conclusion

3.9.5 Overall Performance

The results below are obtained by measuring the execution time of different implementations of modular reduction:

- **Naive scale:** The base operation (%)
- **Shoup scale (reference) :** Shoup's Modular Multiplication without optimization
- **Shoup scale (FLINT) :** the library FLINT's modular reduction implmentation
- **Shoup scale (AVX2) :** Shoup's Modular Multiplication with AVX2 optimization
- **Shoup scale (AVX512) :** Shoup's Modular Multiplication with AVX512 optimization

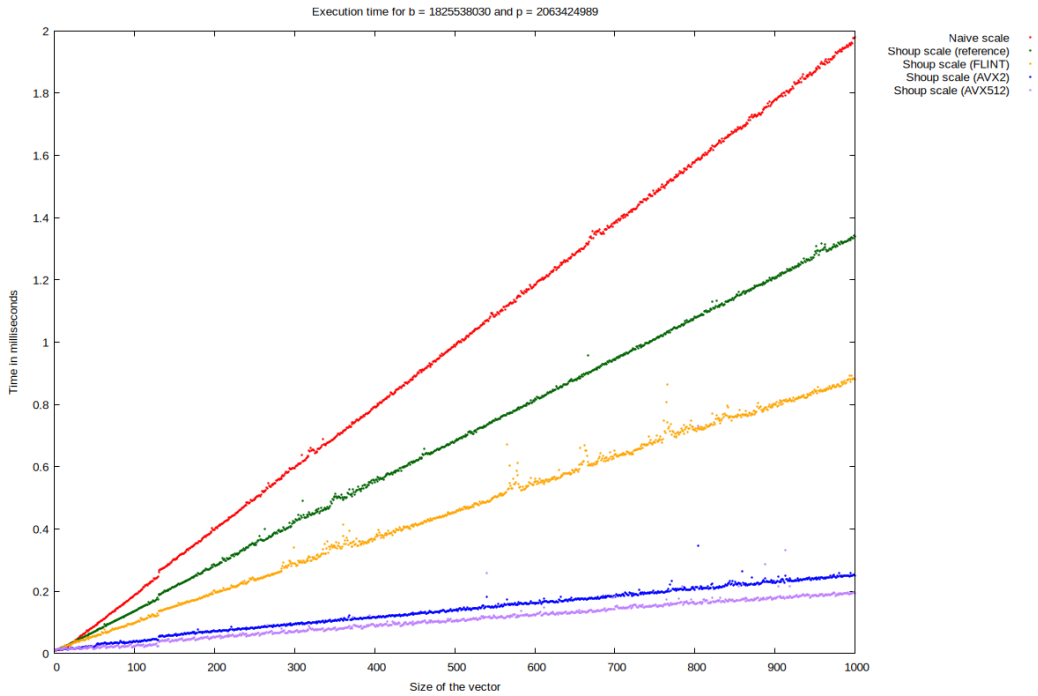

Figure 3: Execution Time Comparison Between Different Implementation For Modular Reduction

Table 5: Execution time for different implementations in relation too the size of the vector in milliseconds

Size	Naive	Shoup ref	FLINT	Shoup AVX2	Shoup AVX512
100	0.187	0.137	0.098	0.037	0.023
200	0.398	0.283	0.201	0.070	0.052
300	0.596	0.422	0.290	0.094	0.072
400	0.794	0.549	0.366	0.116	0.087
500	0.986	0.684	0.455	0.138	0.105
600	1.183	0.816	0.551	0.159	0.123
700	1.383	0.945	0.637	0.184	0.183
800	1.578	1.081	0.718	0.204	0.159
900	1.778	1.204	0.801	0.228	0.179
1000	1.979	1.339	0.880	0.249	0.193

Table 6: Average percentage speedup of different algorithms compared to the naive solution

Size	Naive	Shoup ref	FLINT	Shoup AVX2	Shoup AVX512
100	100%	73.26%	52.41%	19.79%	12.30%
200	100%	71.11%	50.50%	17.59%	13.07%
300	100%	70.81%	48.66%	15.77%	12.08%
400	100%	69.14%	46.10%	14.61%	10.96%
500	100%	69.37%	46.15%	14.00%	10.65%
600	100%	68.98%	46.58%	13.44%	10.40%
700	100%	68.33%	46.06%	13.30%	13.23%
800	100%	68.50%	45.50%	12.93%	10.08%
900	100%	67.72%	45.05%	12.82%	10.07%
1000	100%	67.66%	44.47%	12.58%	9.75%

Table 7: Relative Performance Summary in Percentages

Algorithm	Average %	1x
Naive	100.0%	Baseline
Shoup ref	68.7%	1.45x
FLINT	45.6%	2.2x
Shoup AVX2	15.4%	6.5x
Shoup AVX512	11.4%	8.7x

Observation: From the results retrieved, we can observe that FLINT's implementation over 64 bits provides roughly 2x faster modular reduction on 32 bit operations. Furthermore, our Shoup AVX2 and Shoup AVX512 implementations taillored to 32 bits integers provides significant faster 6-8x speedup.

Conclusion: Our Shoup's Modular Multiplication with AVX2 and AVX512 optimization adapted to 32 bits is 2-3 times faster than FLINT

3.9.6 NEON's Performance

TODO

3.10 Overall Conclusion:

3.11 Other Improvements: